# Striver SDE Sheet - Recursion & Backtracking Revision Notes

*Short DSA revision notes for Recursion & Backtracking problems*

Use this as a quick last-minute revision sheet. Focus on the pattern, decision rule, and complexity.

## Core Patterns

| Pattern | Used In |
|---|---|
| Try every option + undo | Permutations, N-Queens, Sudoku |
| Safety check before recursion | N-Queens, Sudoku, M-Coloring |
| Visited marking | Rat in a Maze |
| Prefix recursion | Word Break |

## Problem-wise Notes

### 1. Print All Permutations of String/Array

| | |
|---|---|
| **Problem** | Generate all possible permutations. |
| **Approach** | Either use a visited array or use in-place swapping. Both fix one position at a time. |
| **Steps** | 1. Visited approach: choose any unused element, mark it, recurse, then unmark.<br>2. Swap approach: swap current index with every possible index, recurse, then swap back.<br>3. When current size/index reaches n, store permutation. |
| **Key Idea** | Fix one position, explore all choices, then undo the choice. |
| **Complexity** | Time: O(n! * n), Space: O(n) |

### 2. N-Queens Problem

| | |
|---|---|
| **Problem** | Place N queens on an N x N board so no two queens attack each other. |
| **Approach** | Place queens column by column. Use row and diagonal arrays for fast safety checks. |
| **Steps** | 1. Try every row in the current column.<br>2. Check leftRow, upperDiagonal, and lowerDiagonal.<br>3. If safe, place queen and mark arrays.<br>4. Recurse for next column.<br>5. Backtrack by removing queen and unmarking arrays. |
| **Key Idea** | Use arrays to check attacks in O(1) instead of scanning the board. |
| **Complexity** | Time: O(n!), Space: O(n^2) for board |

### 3. Sudoku Solver

| | |
|---|---|
| **Problem** | Fill the Sudoku board correctly using digits 1 to 9. |
| **Approach** | Find an empty cell and try every digit. Place only if valid in row, column, and 3x3 box. |
| **Steps** | 1. Find an empty cell.<br>2. Try digits 1 to 9.<br>3. Check row, column, and box validity.<br>4. Place digit and recurse.<br>5. If recursion fails, remove digit and try next. |
| **Key Idea** | Try a digit, continue if valid, undo if it leads to failure. |
| **Complexity** | Time: O(9^emptyCells), Space: recursion stack |

### 4. M-Coloring Problem

| | |
|---|---|
| **Problem** | Color a graph using at most M colors so adjacent nodes have different colors. |
| **Approach** | Assign colors node by node. For each node, try all colors and recurse only if the color is safe. |
| **Steps** | 1. Start from node 0.<br>2. Try colors from 1 to M.<br>3. Check if adjacent nodes already have the same color.<br>4. If safe, assign color and recurse.<br>5. Backtrack if needed. |

| | |
|---|---|
| **Key Idea** | Reject invalid colors early before going deeper. |
| **Complexity** | Time: O(M^N), Space: O(N) |

### 5. Rat in a Maze

| | |
|---|---|
| **Problem** | Find all paths from top-left to bottom-right in a maze. |
| **Approach** | Use backtracking with directions D, L, R, U. Mark cells visited while exploring. |
| **Steps** | 1. Start from cell (0,0).<br>2. If destination is reached, store path.<br>3. Mark current cell visited.<br>4. Try valid moves in allowed directions.<br>5. Recurse and then unmark while returning. |
| **Key Idea** | Visited marking avoids cycles; unmarking allows other paths to use the cell. |
| **Complexity** | Time: O(4^(n*n)), Space: O(n*n) |

### 6. Word Break - Print All Ways

| | |
|---|---|
| **Problem** | Print all valid sentences that can be formed using dictionary words. |
| **Approach** | Try every prefix. If the prefix is in the dictionary, recurse for the remaining string. |
| **Steps** | 1. Start from index 0.<br>2. Try every substring starting from index.<br>3. If substring exists in dictionary, add it to current sentence.<br>4. Recurse for the remaining string.<br>5. Backtrack after recursion. |
| **Key Idea** | Every valid prefix can start one or more sentence formations. |
| **Complexity** | Time: Exponential, Space: O(n) excluding output |

## Final Memory Line

**Backtracking means: try a choice, check validity, recurse, and undo the choice.**